



Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	71451430
Nama Lengkap	Okky Alexander
Minggu ke / Materi	14 / Tipe Data Set

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

MATERI PENGENALAN DAN MENDEFINISIKAN SET

Set merupakan salah satu tipe data bawaan Python yang berfungsi untuk menyimpan sekumpulan nilai yang bersifat unik, di mana tidak ada dua elemen dengan nilai yang sama di dalamnya. Dalam dunia pemrograman, konsep ini sangat erat kaitannya dengan pengertian himpunan dalam matematika. Setiap nilai yang terdapat di dalam Set disebut sebagai anggota atau member dari himpunan tersebut.

Tipe data Set pada Python memiliki beberapa karakteristik khusus yang membedakannya dari tipe data lain. Pertama, seluruh anggota Set wajib bersifat immutable, artinya nilainya tidak dapat berubah setelah dibuat. Contoh tipe data yang bersifat immutable antara lain integer, float, string, dan tuple. Sebaliknya, tipe data mutable seperti list dan dictionary tidak diperkenankan menjadi anggota Set. Kedua, meskipun anggotanya harus immutable, Set itu sendiri bersifat mutable, sehingga isinya masih dapat ditambah maupun dikurangi sewaktu-waktu. Oleh karena sifat mutable inilah, sebuah Set tidak bisa dimasukkan ke dalam Set lain sebagai anggotanya.

Untuk mendefinisikan sebuah Set, terdapat dua cara yang dapat digunakan dalam Python. Cara pertama adalah dengan menggunakan notasi kurung kurawal ({}), yang langsung memuat nilai-nilai anggotanya. Cara kedua adalah dengan memanggil fungsi bawaan `set()` dan meneruskan data yang ingin dijadikan Set sebagai argumennya. Perlu diperhatikan secara khusus bahwa untuk membuat Set yang kosong, notasi {} tidak dapat digunakan karena Python akan menafsirkannya sebagai dictionary kosong, bukan Set kosong. Satu-satunya cara yang benar untuk mendefinisikan Set kosong adalah dengan menggunakan fungsi `set()` tanpa argumen apapun. Contoh penerapan kedua cara tersebut dapat dilihat pada Gambar 14.1 berikut ini.

```
# Cara 1: Menggunakan kurung kurawal {}
bilangan_genap      = {2, 4, 6, 8, 10, 12}
bilangan_ganjil     = {1, 3, 5, 7, 9, 11}

# Cara 2: Menggunakan fungsi set()
pernah_ke_bulan     = set(['Neil Armstrong', 'Buzz Aldrin'])

# Set kosong: wajib menggunakan set()
pernah_ke_mars      = set() # set kosong
data = {}              # ini dictionary, bukan set!
```

Gambar 14.1 Cara mendefinisikan Set menggunakan kurung kurawal dan fungsi `set()`

MATERI PENGAKSESAN SET

Berbeda dengan list maupun tuple, Set tidak memiliki sistem indeks sehingga pengaksesan anggota secara langsung menggunakan nomor posisi tidak dapat dilakukan. Meskipun demikian, kita tetap dapat mengetahui jumlah total elemen yang ada di dalam Set menggunakan fungsi `len()`, serta menelusuri seluruh isinya satu per satu dengan menggunakan perulangan `for`. Apabila program dijalankan, urutan tampilan elemen yang dihasilkan kemungkinan besar tidak akan sesuai dengan urutan saat Set pertama kali dideklarasikan. Hal ini merupakan perilaku yang normal dan wajar, mengingat Set memang tidak mengenal konsep urutan maupun posisi anggota. Yang menjadi perhatian dalam Set hanyalah keberadaan suatu nilai, bukan posisi atau urutannya. Gambar 14.2 menampilkan contoh cara mengakses elemen-elemen di dalam sebuah Set.

```
nim = { '71200120' , '71200195' , '71200214' }

# Menghitung jumlah elemen
jumlah_nim = len ( nim )
print ( jumlah_nim )      # output: 3

# Iterasi seluruh elemen set
for n in nim :
    print ( n )

# Urutan output tidak selalu sama dengan deklarasi
```

Gambar 14.2 Pengaksesan elemen Set menggunakan fungsi `len()` dan perulangan `for`

Karena Set bersifat mutable, penambahan elemen baru ke dalam Set dapat dilakukan kapan saja menggunakan metode `add()`. Salah satu keunggulan metode ini adalah kemampuannya melakukan pemeriksaan duplikasi secara otomatis tanpa perlu pengecekan manual dari programmer. Ketika sebuah nilai baru hendak ditambahkan, Python akan terlebih dahulu memeriksa apakah nilai tersebut sudah ada di dalam Set. Bila belum ada, nilai tersebut akan berhasil masuk ke dalam Set. Namun apabila nilai yang sama sudah terdapat di dalamnya, pemanggilan `add()` tidak akan menghasilkan perubahan apapun dan tidak memunculkan pesan kesalahan.

Untuk penghapusan elemen dari Set, Python menyediakan empat metode dengan perilaku yang sedikit berbeda satu sama lain. Metode `discard()` menghapus elemen tertentu yang disebutkan tanpa memunculkan error bila elemen tidak ditemukan di dalam Set. Metode `remove()` juga menghapus elemen tertentu, namun akan memunculkan error jika elemen yang dimaksud tidak ada. Metode `pop()` bekerja berbeda, yakni mengambil dan sekaligus menghapus satu elemen secara acak dari Set dan akan error jika

Set dalam kondisi kosong. Terakhir, metode `clear()` akan mengosongkan seluruh isi Set sekaligus tanpa memunculkan error. Contoh penggunaan metode penambahan dan penghapusan tersebut dapat dilihat pada Gambar 14.3.

```
plat_nomor      = set ( )

plat_nomor      . add ( 'AB 1890 XA' )      # menambah elemen

plat_nomor      . add ( 'AD 6810 MT' )

plat_nomor      . add ( 'AB 1890 XA' )      # duplikat, diabaikan

print ( len ( plat_nomor ) )                # output: 2


bilangan_prima  = { 13 , 23 , 7 , 29 , 11 , 5 }

bilangan_prima  . remove ( 5 )              # hapus elemen 5

bilangan_prima  . discard ( 97 )            # 97 tidak ada, tidak error

bilangan        = bilangan_prima . pop ( )  # ambil & hapus secara acak

bilangan_prima  . clear ( )                # kosongkan seluruh set
```

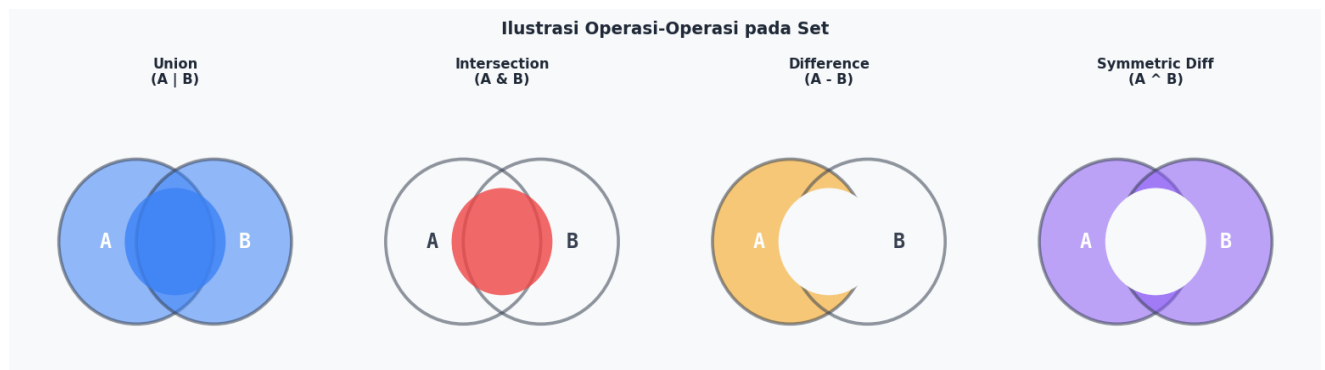
Gambar 14.3 Contoh penggunaan metode `add()`, `remove()`, `discard()`, `pop()`, dan `clear()` pada Set

Apabila ingin mengubah nilai suatu anggota yang sudah ada di dalam Set, hal tersebut tidak dapat dilakukan secara langsung karena Python tidak menyediakan mekanisme modifikasi nilai elemen Set. Cara yang dapat ditempuh adalah dengan menghapus terlebih dahulu anggota lama yang ingin diganti menggunakan `remove()` atau `discard()`, kemudian memasukkan nilai baru yang diinginkan menggunakan `add()`. Proses ini dikenal sebagai operasi penggantian atau replace. Setiap kali terjadi penambahan atau penghapusan, susunan tampilan elemen di dalam Set dapat berubah secara dinamis, yang kembali menegaskan bahwa posisi elemen pada Set tidaklah relevan.

MATERI OPERASI OPERASI SET

Layaknya himpunan dalam matematika, tipe data Set pada Python mendukung berbagai operasi relasional yang dapat diterapkan pada dua himpunan atau lebih. Operasi-operasi ini sangat berguna dalam pemrograman ketika kita perlu membandingkan, menggabungkan, atau menyaring data dari dua kelompok yang berbeda. Terdapat empat operasi utama yang tersedia pada tipe data Set, yaitu union,

intersection, difference, dan symmetric difference. Gambaran visual dari keempat operasi tersebut dapat dilihat pada diagram Venn di Gambar 14.4 berikut ini.



Gambar 14.4 Ilustrasi diagram Venn dari keempat operasi pada Set

Operasi union digunakan untuk menggabungkan seluruh elemen dari dua Set menjadi satu himpunan baru yang memuat semua anggota dari kedua Set tersebut. Apabila terdapat elemen yang sama di kedua Set, elemen tersebut hanya akan muncul satu kali dalam hasil gabungan, sesuai dengan prinsip keunikan yang menjadi dasar tipe data Set. Dalam Python, operasi union dapat dilakukan menggunakan operator `|` ataupun dengan memanggil metode `union()` pada salah satu Set dengan Set lainnya sebagai argumen.

Operator Intersection

Operasi intersection bertujuan untuk menemukan elemen-elemen yang menjadi anggota dari dua Set secara bersamaan, atau dengan kata lain mencari irisan dari dua himpunan yang diberikan. Hanya elemen yang terdapat di kedua Set sekaligus yang akan masuk ke dalam hasil operasi ini. Elemen yang hanya ada di salah satu Set saja tidak akan diikutsertakan. Dalam Python, operasi ini dapat dilakukan menggunakan operator `&` atau metode `intersection()`. Contoh implementasi operator union dan intersection dapat dilihat bersama pada Gambar 14.5 berikut.

```

merek_hp    = { 'Samsung' , 'Apple' , 'Xiaomi' , 'Sony' }
merek_ac    = { 'LG' , 'Samsung' , 'Panasonic' , 'Daikin' , 'Sony' }

# Union: gabungan semua elemen
gabungan    = merek_hp | merek_ac
# {'Xiaomi','Sony','LG','Daikin','Samsung','Apple','Panasonic'}

renang      = { 'siti' , 'mail' , 'ikhshan' , 'upin' , 'ipin' }
tenis       = { 'joko' , 'mail' , 'ipin' , 'upin' , 'tejo' }

# Intersection: elemen yang ada di kedua set sekaligus
renang_tenis = renang & tenis
print ( renang_tenis ) # output: {'upin', 'ipin', 'mail'}

```

Gambar 14.5 Contoh penggunaan operator union (|) dan intersection (&) pada Set

Operator Difference

Operasi difference menghasilkan himpunan baru yang memuat elemen-elemen yang hanya terdapat di Set pertama dan tidak dimiliki oleh Set kedua. Penting untuk dipahami bahwa hasil operasi difference sangat bergantung pada urutan Set yang dibandingkan, karena $A - B$ belum tentu menghasilkan nilai yang sama dengan $B - A$. Dalam Python, operasi ini dapat dilakukan menggunakan operator minus (-) atau metode difference().

Operator Symmetric Difference

Operasi symmetric difference menghasilkan himpunan baru yang memuat semua elemen dari kedua Set, kecuali elemen-elemen yang menjadi irisan keduanya. Dengan kata lain, hasilnya hanya memuat elemen yang bersifat eksklusif di masing-masing Set dan tidak dimiliki bersama. Dalam Python, operasi ini menggunakan operator ^ atau metode symmetric_difference(). Secara matematis, operasi ini setara dengan melakukan union terlebih dahulu kemudian mengurangnya dengan hasil intersection dari kedua Set yang sama. Gambar 14.6 menampilkan contoh penerapan operator difference dan symmetric difference.

```

english = {'desi' , 'tono' , 'evan' , 'miko' , 'takashi' , 'chaewon' }
korean  = {'chaewon' , 'yeona' , 'erika' , 'miko' }

# Difference: elemen eksklusif salah satu set
only_korean = korean - english
print ( only_korean )      # output: {'erika', 'yeona'}

only_english = english - korean
print ( only_english )     # output: {'takashi', 'desi', 'evan', 'tono'}

# Symmetric Difference: gabungan minus irisan
one_language = english ^ korean
print ( one_language )     # {'yeona','tono','desi','erika','evan','takashi'}

```

Gambar 14.6 Contoh penggunaan operator difference (-) dan symmetric difference (^) pada Set

Keempat operasi himpunan di atas menjadikan tipe data Set sebagai alat yang sangat powerful dan efisien dalam menangani berbagai permasalahan yang berkaitan dengan pengelompokan data, perbandingan antar kelompok, serta penyaringan elemen secara logis. Kemampuan ini membuat Set menjadi pilihan yang tepat dalam berbagai skenario pemrograman Python, mulai dari analisis data sederhana hingga implementasi algoritma yang lebih kompleks.

Source Github : https://github.com/tuangkeman/71451430_Okky.git

LATIHAN 14.1

```
n = int(input('Masukkan jumlah kategori: '))

data_aplikasi = {}

for i in range(n):
    nama_kategori = input('Masukkan nama kategori: ')
    print('Masukkan 5 nama aplikasi di kategori', nama_kategori)
    aplikasi = []
    for j in range(5):
        nama_aplikasi = input('Nama aplikasi: ')
        aplikasi.append(nama_aplikasi)

    data_aplikasi[nama_kategori] = aplikasi

print('\n--- Daftar Aplikasi per Kategori ---')
for kategori, aplikasi in data_aplikasi.items():
    print(f'{kategori}: {aplikasi}')

daftar_aplikasi_set = []
for aplikasi in data_aplikasi.values():
    daftar_aplikasi_set.append(set(aplikasi))

hasil_semua = daftar_aplikasi_set[0]
for i in range(1, len(daftar_aplikasi_set)):
    hasil_semua = hasil_semua.intersection(daftar_aplikasi_set[i])

print('\n--- Aplikasi yang muncul di SEMUA kategori ---')
print(hasil_semua if hasil_semua else 'Tidak ada aplikasi yang muncul di semua kategori')

print('\n--- Aplikasi yang hanya muncul di SATU kategori saja ---')
kategori_list = list(data_aplikasi.keys())
for idx, (kategori, aplikasi_set) in enumerate(zip(kategori_list, daftar_aplikasi_set)):
    set_lain = set()
    for j, s in enumerate(daftar_aplikasi_set):
        if j != idx:
            set_lain = set_lain.union(s)
    # aplikasi yang hanya ada di kategori ini
    hanya_disini = aplikasi_set - set_lain
    print(f'{kategori}: {hanya_disini}')

if n > 2:
    print('\n--- Aplikasi yang muncul tepat di DUA kategori ---')
    muncul_dua = set()
    for i in range(len(daftar_aplikasi_set)):
        for j in range(i + 1, len(daftar_aplikasi_set)):
            irisan_dua = daftar_aplikasi_set[i].intersection(daftar_aplikasi_set[j])

            for k in range(len(daftar_aplikasi_set)):
                if k != i and k != j:
                    irisan_dua = irisan_dua - daftar_aplikasi_set[k]
            if irisan_dua:
                print(f'{kategori_list[i]} & {kategori_list[j]}: {irisan_dua}')
                muncul_dua = muncul_dua.union(irisan_dua)
    if not muncul_dua:
        print('Tidak ada aplikasi yang muncul tepat di dua kategori')
```

Gambar 14.7 Kode latihan 14.1

Latihan 14.1 merupakan pengembangan dari Contoh 14.3 yang membahas kategori aplikasi di Play Store. Pada latihan ini, program yang sudah ada diperluas dengan dua kemampuan tambahan, yaitu menampilkan aplikasi yang hanya muncul di satu kategori saja, serta menampilkan aplikasi yang muncul tepat di dua kategori sekaligus khusus untuk kondisi n lebih dari 2.

Untuk menampilkan aplikasi yang hanya muncul di satu kategori saja, program memanfaatkan operasi difference (selisih himpunan). Untuk setiap kategori, program menggabungkan semua set dari kategori lain menggunakan operasi union, lalu mengurangi set kategori tersebut dengan gabungan tadi. Hasilnya adalah anggota-anggota yang benar-benar eksklusif hanya ada di satu kategori itu saja.

Untuk menampilkan aplikasi yang muncul tepat di dua kategori, program menggunakan kombinasi intersection berpasangan. Setiap pasangan kategori (i, j) dicari irisannya terlebih dahulu, kemudian hasil irisan tersebut dikurangi dengan anggota dari semua kategori lain agar hanya tersisa aplikasi yang benar-benar muncul di dua kategori saja, tidak lebih. Fitur ini hanya aktif ketika jumlah kategori (n) lebih dari 2.

Program ini mendemonstrasikan bahwa operasi-operasi himpunan pada Set bisa dikombinasikan secara fleksibel untuk menjawab berbagai pertanyaan analitis pada data, tidak hanya satu jenis operasi saja.

Ouput :

```
Masukkan jumlah kategori: 3
Masukkan nama kategori: game
Masukkan 5 nama aplikasi di kategori game
Nama aplikasi: ml
Nama aplikasi: pubg
Nama aplikasi: nte
Nama aplikasi: wuwa
Nama aplikasi: stokbit
Masukkan nama kategori: saham
Masukkan 5 nama aplikasi di kategori saham
Nama aplikasi: mexc
Nama aplikasi: stokbit
Nama aplikasi: ajaib
Nama aplikasi: bri
Nama aplikasi: bca
Masukkan nama kategori: paylater
Masukkan 5 nama aplikasi di kategori paylater
Nama aplikasi: bca
Nama aplikasi: bri
Nama aplikasi: shope
Nama aplikasi: dana
Nama aplikasi: ovo

=== Daftar Aplikasi per Kategori ===
game: ['ml', 'pubg', 'nte', 'wuwa', 'stokbit']
saham: ['mexc', 'stokbit', 'ajaib', 'bri', 'bca']
paylater: ['bca', 'bri', 'shope', 'dana', 'ovo']

Aplikasi yang muncul di SEMUA kategori
Tidak ada aplikasi yang muncul di semua kategori

Aplikasi yang hanya muncul di SATU kategori saja
game: {'wuwa', 'ml', 'pubg', 'nte'}
saham: {'mexc', 'ajaib'}
paylater: {'dana', 'shope', 'ovo'}

Aplikasi yang muncul tepat di DUA kategori
game & saham: {'stokbit'}
saham & paylater: {'bca', 'bri'}
```

Gambar 14.8 Output Program 14.1

LATIHAN 14.2

```
print(' 1. List menjadi Set ')
data_list = [1, 2, 3, 2, 4, 1, 5]
print(f'Sebelum (List) : {data_list}')
data_set = set(data_list)
print(f'Sesudah (Set) : {data_set}')

print('\n 2. Set menjadi List ')
data_set2 = {10, 20, 30, 40, 50}
print(f'Sebelum (Set) : {data_set2}')
data_list2 = list(data_set2)
print(f'Sesudah (List) : {data_list2}')

print('\n 3. Tuple menjadi Set ')
data_tuple = (7, 7, 8, 9, 9, 10)
print(f'Sebelum (Tuple): {data_tuple}')
data_set3 = set(data_tuple)
print(f'Sesudah (Set) : {data_set3}')

print('\n 4. Set menjadi Tuple ')
data_set4 = {'apel', 'mangga', 'jeruk', 'pisang'}
print(f'Sebelum (Set) : {data_set4}')
data_tuple2 = tuple(data_set4)
print(f'Sesudah (Tuple): {data_tuple2}')
```

Gambar 14.9 Kode latihan 14.2

Latihan 14.1 merupakan pengembangan dari Contoh 14.3 yang membahas kategori aplikasi di Play Store. Pada latihan ini, program yang sudah ada diperluas dengan dua kemampuan tambahan, yaitu menampilkan aplikasi yang hanya muncul di satu kategori saja, serta menampilkan aplikasi yang muncul tepat di dua kategori sekaligus khusus untuk kondisi n lebih dari 2.

Untuk menampilkan aplikasi yang hanya muncul di satu kategori saja, program memanfaatkan operasi difference (selisih himpunan). Untuk setiap kategori, program menggabungkan semua set dari kategori lain menggunakan operasi union, lalu mengurangi set kategori tersebut dengan gabungan tadi. Hasilnya adalah anggota-anggota yang benar-benar eksklusif hanya ada di satu kategori itu saja.

Untuk menampilkan aplikasi yang muncul tepat di dua kategori, program menggunakan kombinasi intersection berpasangan. Setiap pasangan kategori (i, j) dicari irisannya terlebih dahulu, kemudian hasil irisan tersebut dikurangi dengan anggota dari semua kategori lain agar hanya tersisa aplikasi yang benar-benar muncul di dua kategori saja, tidak lebih. Fitur ini hanya aktif ketika jumlah kategori (n) lebih dari 2.

Program ini mendemonstrasikan bahwa operasi-operasi himpunan pada Set bisa dikombinasikan secara fleksibel untuk menjawab berbagai pertanyaan analitis pada data, tidak hanya satu jenis operasi saja.

Output :

```

1. List menjadi Set
Sebelum (List) : [1, 2, 3, 2, 4, 1, 5]
Sesudah (Set)  : {1, 2, 3, 4, 5}

2. Set menjadi List
Sebelum (Set)  : {50, 20, 40, 10, 30}
Sesudah (List) : [50, 20, 40, 10, 30]

3. Tuple menjadi Set
Sebelum (Tuple): (7, 7, 8, 9, 9, 10)
Sesudah (Set)  : {8, 9, 10, 7}

4. Set menjadi Tuple
Sebelum (Set)  : {'apel', 'mangga', 'pisang', 'jeruk'}
Sesudah (Tuple): ('apel', 'mangga', 'pisang', 'jeruk')

```

Gambar 14.10 Output Program 14.2

LATIHAN 14.3

```

def baca_kata(nama_file):
    try:
        with open(nama_file, 'r') as f:
            isi = f.read()

            kata_kata = set()
            for kata in isi.split():
                kata_bersih = kata.strip('.,!?:\'"()[]').lower()
                if kata_bersih:
                    kata_kata.add(kata_bersih)
            return kata_kata
    except FileNotFoundError:
        print(f'Error: File "{nama_file}" tidak ditemukan!')
        return None
    except Exception as e:
        print(f'Error: Tidak bisa membaca file "{nama_file}": {e}')
        return None

nama_file1 = input('Masukkan nama file pertama: ')
nama_file2 = input('Masukkan nama file kedua: ')

kata_file1 = baca_kata(nama_file1)
kata_file2 = baca_kata(nama_file2)

if kata_file1 is not None and kata_file2 is not None:
    kata_sama = kata_file1.intersection(kata_file2)

    print(f'\n Kata-kata di "{nama_file1}" ')
    print(sorted(kata_file1))

    print(f'\n Kata-kata di "{nama_file2}" ')
    print(sorted(kata_file2))

    print(f'\n Kata yang muncul di KEDUA file ')
    print(sorted(kata_sama))

```

Gambar 14.11 Output Program 14.3

Latihan 14.3 mengaplikasikan konsep Set dalam konteks pengolahan file teks. Program meminta pengguna untuk memasukkan nama dua buah file teks, kemudian membaca isi kedua file tersebut dan menampilkan semua kata yang muncul pada kedua file sekaligus.

Proses pembacaan file dilakukan di dalam sebuah fungsi bernama `baca_kata()` yang menerima nama file sebagai parameter. Di dalam fungsi ini, file dibaca menggunakan blok `try-except` untuk menangani

kemungkinan error, seperti ketika file tidak ditemukan (FileNotFoundError) atau ketika file tidak bisa dibaca karena alasan lain. Jika terjadi error, program akan menampilkan pesan yang informatif kepada pengguna daripada berhenti secara tiba-tiba.

Setelah file berhasil dibaca, setiap kata dalam file diproses dengan cara dipecah menggunakan metode `split()`, kemudian tanda baca seperti titik, koma, tanda seru, dan sebagainya dihilangkan dari awal dan akhir kata menggunakan metode `strip()`. Seluruh kata juga dikonversi menjadi huruf kecil (lowercase) menggunakan metode `lower()` agar pencocokan kata tidak bersifat case-sensitive. Misalnya kata "Python" dan "python" akan dianggap sebagai kata yang sama.

Setelah kedua file diproses menjadi dua buah Set kata, program menggunakan operasi intersection untuk menemukan kata-kata yang muncul di kedua file sekaligus. Program kemudian menampilkan daftar kata dari masing-masing file secara terurut (menggunakan `sorted()`) serta daftar kata yang sama-sama muncul di kedua file. Latihan ini merupakan contoh nyata penggunaan Set untuk menyelesaikan masalah perbandingan data teks secara efisien.

Output :

```
Masukkan nama file pertama: E:\Koolyeh\github\PrakAlPro-71251230\Minggu 14\file1.txt
Masukkan nama file kedua: E:\Koolyeh\github\PrakAlPro-71251230\Minggu 14\file2.txt

Kata-kata di "E:\Koolyeh\github\PrakAlPro-71251230\Minggu 14\file1.txt"
['a', 'and', 'data', 'developers', 'development', 'easy', 'for', 'functional', 'great', 'is', 'it', 'language', 'learn', 'love', 'many', 'object', 'oriented', 'powerful', 'programming', 'python', 'science', 'supports', 'to', 'use', 'web']

Kata-kata di "E:\Koolyeh\github\PrakAlPro-71251230\Minggu 14\file2.txt"
['a', 'also', 'and', 'applications', 'developers', 'development', 'feature', 'for', 'is', 'it', 'java', 'key', 'language', 'many', 'mobile', 'object', 'of', 'oriented', 'popular', 'programming', 'python', 'together', 'use', 'used', 'web']

Kata yang muncul di KEDUA file
['a', 'and', 'developers', 'development', 'for', 'is', 'it', 'language', 'many', 'object', 'oriented', 'programming', 'python', 'use', 'web']
PS E:\Koolyeh\github\PrakAlPro-71251230\Minggu 14>
```